

Aula 2 (Módulo 02) — PowerShell

1. O que é PowerShell (visão simples)

Assim como o Bash é o "shell" do Linux (Módulo 00, Aula 4), o **PowerShell** é o shell mais moderno e poderoso do Windows — a ferramenta de linha de comando usada tanto por administradores quanto, infelizmente, por grande parte dos ataques modernos contra Windows.

A diferença central para o `cmd.exe` (o "prompt de comando" mais antigo do Windows) é que no PowerShell **tudo é objeto**, não texto — quando você lista processos, por exemplo, cada processo é um objeto com propriedades (`Name`, `Id`, `CPU` ...) que você pode filtrar, ordenar e combinar diretamente, sem precisar "quebrar texto" como fazíamos com `awk` / `grep` no Linux.

2. Comandos básicos: Cmdlets

Comandos do PowerShell seguem o padrão `Verbo-Substantivo`:

```
Get-Process      # lista processos (usamos na aula anterior)
Get-Service      # lista serviços
Get-ChildItem     # lista arquivos/pastas (equivalente ao "ls" do Linux)
Get-Content arquivo.txt # lê o conteúdo de um arquivo (equivalente ao "cat")
Get-Help Get-Process # ajuda oficial sobre qualquer cmdlet
```

Verbos comuns: `Get` (obter), `Set` (definir), `New` (criar), `Remove` (remover), `Start` / `Stop` (iniciar/parar). Essa padronização torna o PowerShell mais previsível de aprender do que decorar comandos soltos.

3. O pipe (`|`) — igual ao Linux, só que com objetos

```
Get-Process | Where-Object {$_.CPU -gt 100} | Sort-Object CPU -Descending
```

Isso filtra processos com mais de 100 (segundos de CPU acumulados) e ordena do maior consumidor para o menor — o mesmo espírito do `ps -ef | grep ... | sort` que fizemos no Linux, mas operando em propriedades de objeto em vez de texto.

4. Por que PowerShell é tão relevante em segurança ofensiva

PowerShell vem **instalado por padrão** em todo Windows moderno, tem acesso completo ao .NET Framework, consegue baixar e executar código diretamente **na memória** (sem gravar arquivo em disco), e consegue interagir com quase qualquer parte do sistema operacional (Registry, rede, Active Directory...). Essa combinação faz dele uma das ferramentas mais usadas por atacantes depois de conseguir um primeiro acesso — é o que se chama de **"living off the land"**: usar ferramentas legítimas já presentes no sistema, em vez de trazer malware externo, o que dificulta a detecção por antivírus tradicional baseado em assinatura.

Command line ofuscada — o sinal mais clássico

```
powershell.exe -enc SQBFAFgAIAAoAE4AZQB3AC0ATwBiAGoAZQBjAHQA...
```

A flag `-enc` (ou `-EncodedCommand`) recebe um comando em **Base64** (uma forma de codificar texto/binário como texto ASCII — não é criptografia, é só uma transformação reversível) — muito usado por atacantes para dificultar a leitura direta do comando em um log, e para evitar que caracteres especiais quebrem a execução. Um analista bem treinado sabe **decodificar** esse Base64 para ver o comando real (vamos praticar isso quando chegarmos aos módulos de PowerShell logging e detecção).

Outras flags comuns em comandos maliciosos: - `-WindowStyle Hidden` → executa sem mostrar janela - `-ExecutionPolicy Bypass` → ignora a política que normalmente bloqueia a execução de scripts `.ps1` não assinados - `-NoProfile` → não carrega configurações do usuário (mais rápido e discreto)

Por que importa para segurança: essas flags, juntas, formam um dos padrões de detecção mais ensinados em qualquer treinamento de Blue Team — uma linha de comando com `-enc` + `-WindowStyle Hidden` + `-ExecutionPolicy Bypass` tem altíssima chance de ser maliciosa, mesmo sem saber o que o comando decodificado faz. Vamos formalizar essa detecção de verdade quando chegarmos a PowerShell logging (mais adiante neste módulo) e a Sysmon (Módulo 11).

5. Execution Policy — controle, não segurança absoluta

```
Get-ExecutionPolicy
```

O Windows tem uma configuração que restringe quais scripts `.ps1` podem rodar (`Restricted`, `AllSigned`, `RemoteSigned`, `Unrestricted`...). **Importante entender:** isso é um controle de conveniência, não uma barreira de segurança forte — existem formas bem conhecidas e documentadas de contorná-lo (como o próprio `-ExecutionPolicy Bypass` citado acima). Não confie nele como única defesa; ele existe principalmente para evitar execução **acidental** de scripts, não para parar um atacante determinado.

6. Exercício prático

Se você tiver acesso a um Windows, abra o PowerShell e rode:

```
Get-Process | Where-Object {$_.CPU -gt 10} | Sort-Object CPU -Descending | Select-Object Name, Id, CPU -Fi  
Get-ExecutionPolicy
```

Depois, só para praticar leitura de Base64 (sem executar nada perigoso), decodifique este comando de exemplo:

```
$texto = "RwB1AHQALQBQAHIAbwBjAGUAcwBZAA=="  
[System.Text.Encoding]::Unicode.GetString([System.Convert]::FromBase64String($texto))
```

Isso deve revelar o comando original em texto simples — é exatamente o tipo de decodificação que você faria ao investigar um `-enc` suspeito em um log real.

7. Recapitulando

- PowerShell = shell moderno do Windows, baseado em objetos (`Cmdlets` no padrão `Verbo-Substantivo`), com pipe (`|`) equivalente ao Bash.
- É a ferramenta favorita de ataques "living off the land" por vir pré-instalada e ter acesso profundo ao sistema.
- `-EncodedCommand` / `-enc` (Base64), `-WindowStyle Hidden` e `-ExecutionPolicy Bypass` juntos formam um padrão clássico de detecção de execução maliciosa.
- Execution Policy é conveniência, não uma barreira de segurança forte.

Próxima aula

Windows Defender, Windows Firewall, Task Scheduler e WMI — as outras peças nativas do Windows que tanto defendem quanto (se mal usadas ou abusadas) viram vetores de ataque.

Fontes recomendadas

- **Microsoft Learn — "About Cmdlets"**: documentação oficial da estrutura Verbo-Substantivo e do modelo de objetos do PowerShell. <https://learn.microsoft.com/powershell/scripting/developer/cmdlet/cmdlet-overview>
- **Microsoft Learn — "about_Execution_Policies"**: documentação oficial deixando claro que Execution Policy não é um controle de segurança contra usuários mal-intencionados.
https://learn.microsoft.com/powershell/module/microsoft.powershell.core/about/about_execution_policies
- **MITRE ATT&CK — T1059.001 (Command and Scripting Interpreter: PowerShell)**: técnica oficial associada ao uso ofensivo de PowerShell. <https://attack.mitre.org/techniques/T1059/001/>